Being More Professional!

Ehsan Khamespanah

Spring 2026

Collections in Java

Collections in Java

- To support more flexible data structures the core Java library provides the collection framework
- The data stored in the collection is encapsulated and the access to the data is only possible via predefined methods
- Java typically provides an interface, like **List** and one or several implementations for this interface, e.g., **ArrayList**, **LinkedList**

Lists and Maps

```
List<Integer> list = new ArrayList<Integer>();

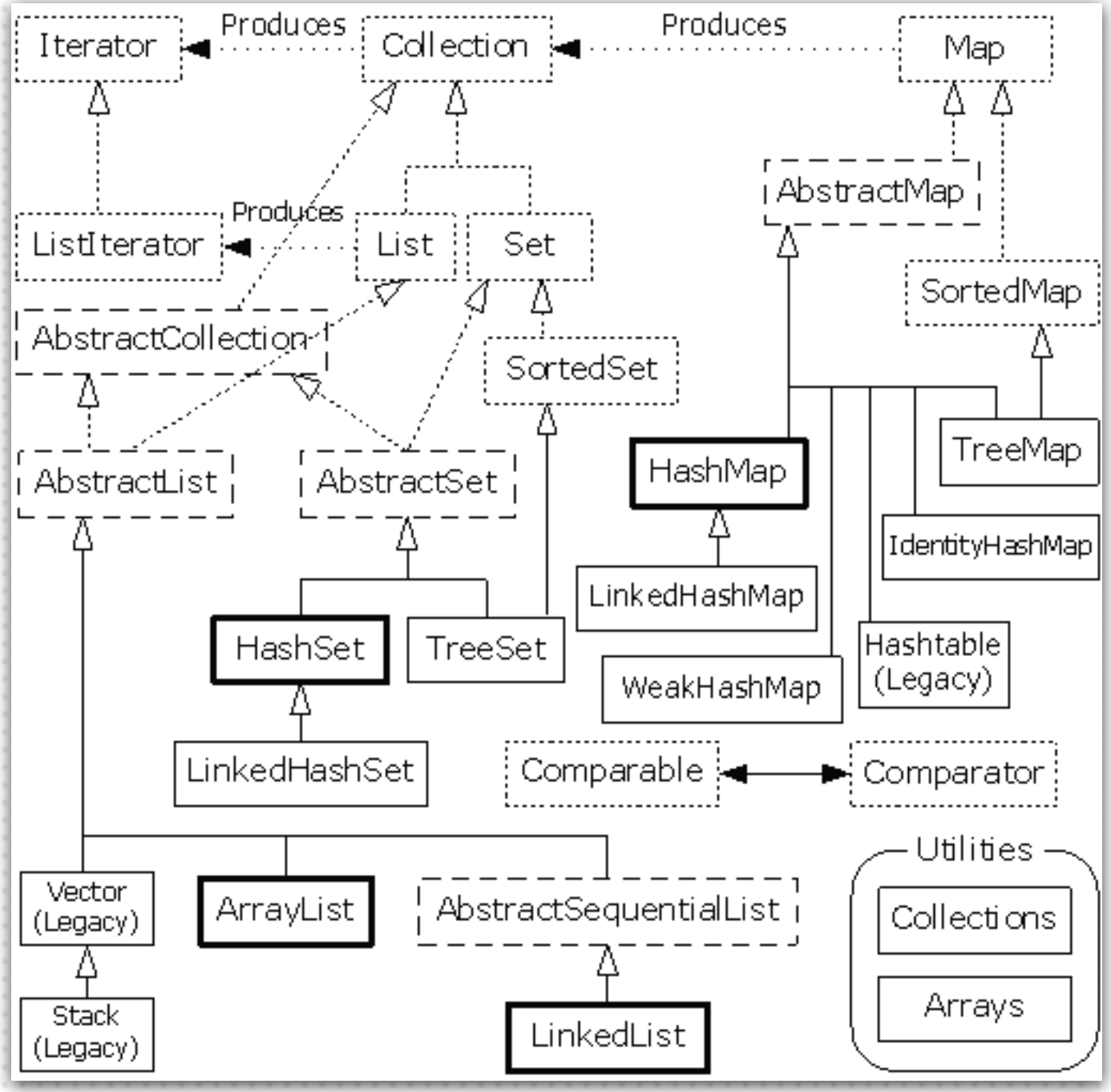
list.add(2);
list.add(3);
list.add(-1);

for (Integer value : list) {
      System.out.println(value);
}
```

```
Map<String, String> map = new HashMap<>();

map.put("Android", "Mobile");
map.put("Eclipse IDE", "Java");
map.put("Eclipse RCP", "Java");
map.put("Git", "Version control system");

map.remove("Android");
```


Iterator
Produces
Collection
Produces
Map
AbstractMap
ListIterator
Produces
List
Set
SortedMap
AbstractCollection
SortedSet
AbstractList
AbstractSet
HashMap
TreeMap
IdentityHashMap
LinkedHashMap
HashSet
TreeSet
WeakHashMap
Hashtable
(Legacy)
LinkedHashSet
Comparable
Comparator
Utilities
Collections
Arrays
Vector
(Legacy)
ArrayList
AbstractSequentialList
Stack
(Legacy)
LinkedList

More Data Structures

- **ConcurrentHashMap**: a thread-safe version of HashMap, Instead of locking the entire map, it locks only specific portions (buckets) for better performance.
- **CopyOnWriteArrayList**: a thread-safe version of ArrayList where a new copy of the list is created whenever it is modified. Best suited for scenarios with frequent reads and few writes.
- **BlockingQueue:** It blocks the producer when the queue is full and blocks the consumer when it is empty.

Introduction to JSON

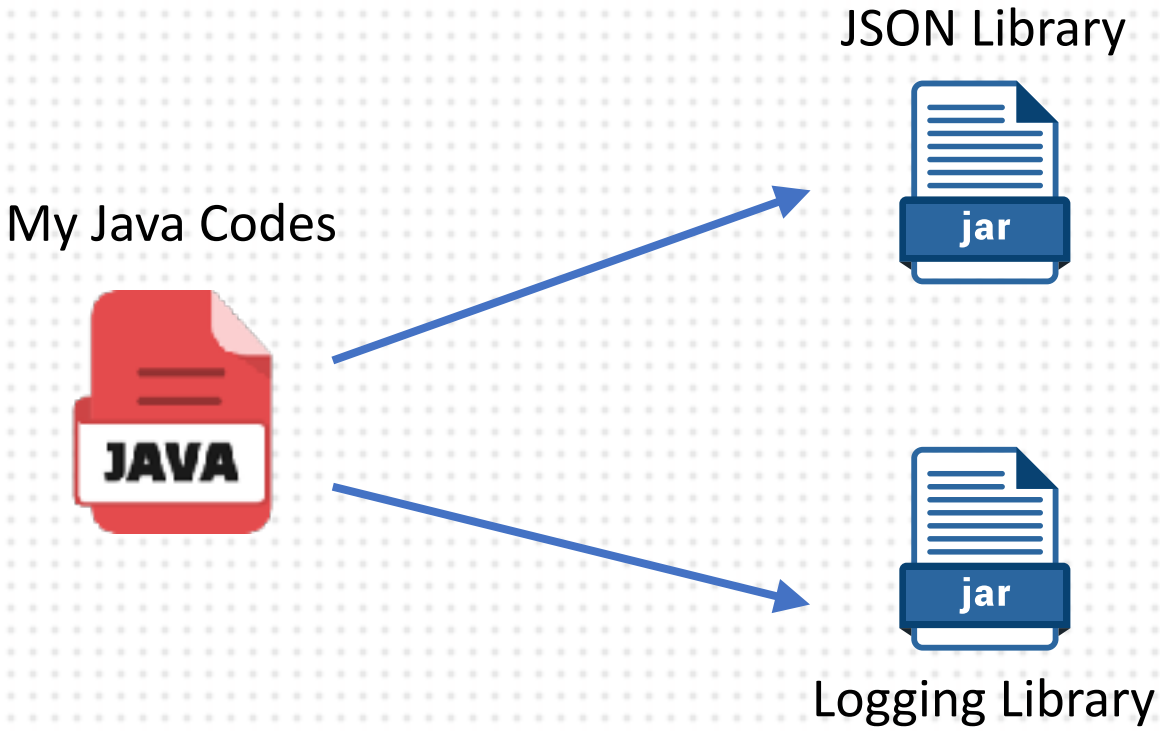
What is JSON?

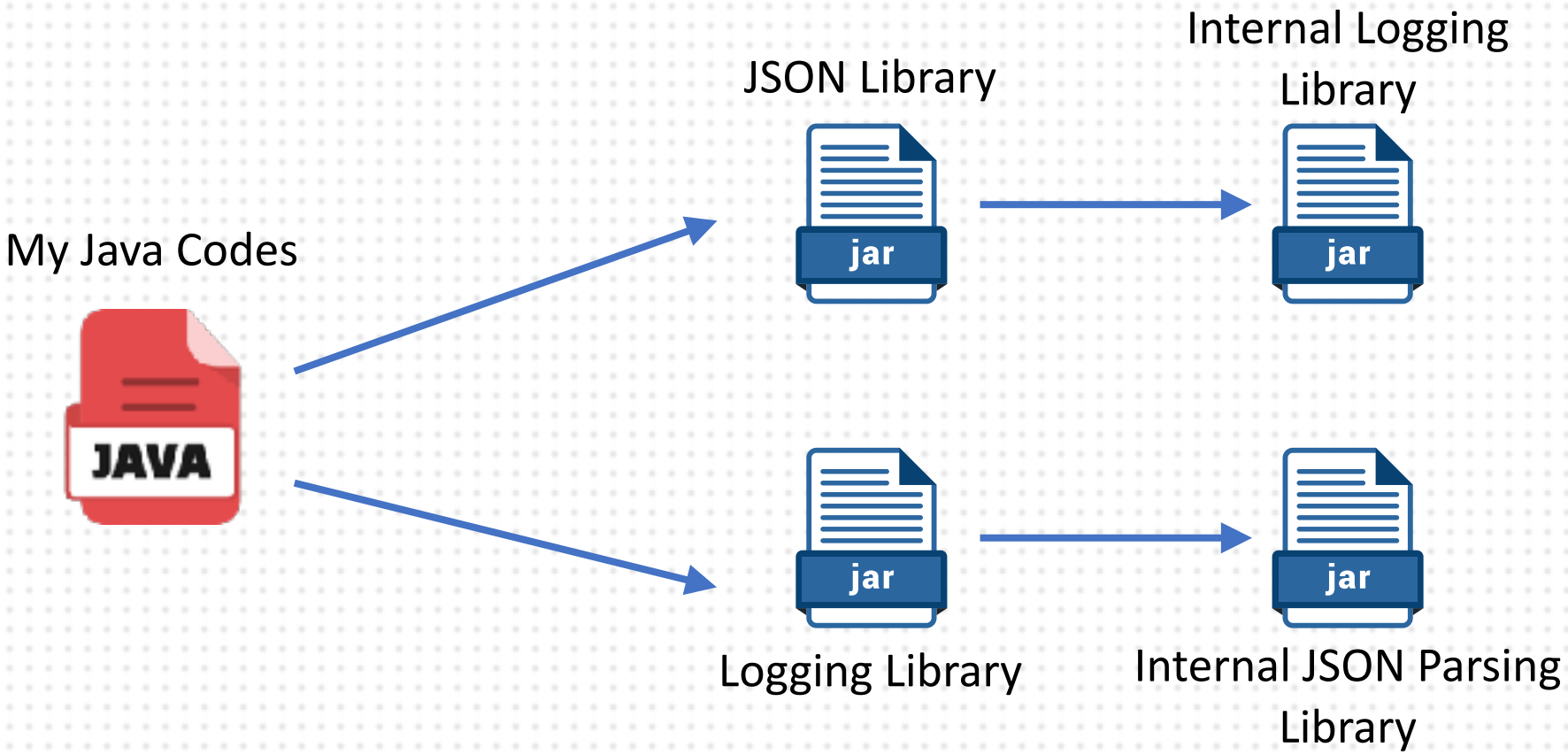
- A human readable data presentation standard
- Data and field names are presented together

```
{
    "from" : "Tehran",
    "to" : "Shiraz",
    "departure" : "10:00",
    "arrival" : "11:00",
    "carrier" : "IRA"
}
```

```
[{
    "from" : "Tehran",
    "to" : "Shiraz",
    "departure" : "10:00",
    "arrival" : "11:00",
    "carrier" : "IRA"
},
{
    "from" : "Beijing",
    "to" : "Tehran",
    "departure" : "12:00",
    "arrival" : "14:00",
    "carrier" : "CSN"
}
]
```

Build and Configuration Management


JSON Library
My Java Codes
JAVA
jar
jar
Logging Library


My Java Codes
JAVA
JSON Library
jar
Internal Logging Library
jar
Logging Library
jar
Internal JSON Parsing Library
jar

Conflicts?

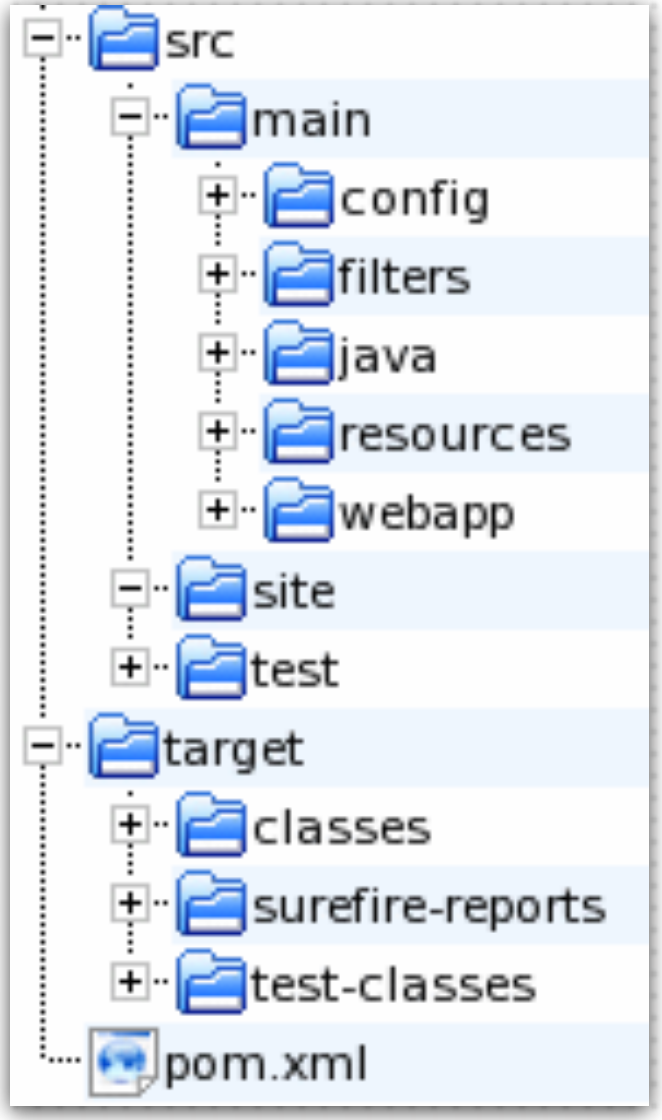
What is Maven?

- A popular open source build tool for enterprise Java projects
- Build and configuration Management
  - Versioning
  - Dependencies
  - Build task
- The project structure and contents are described in a POM (project object model) file

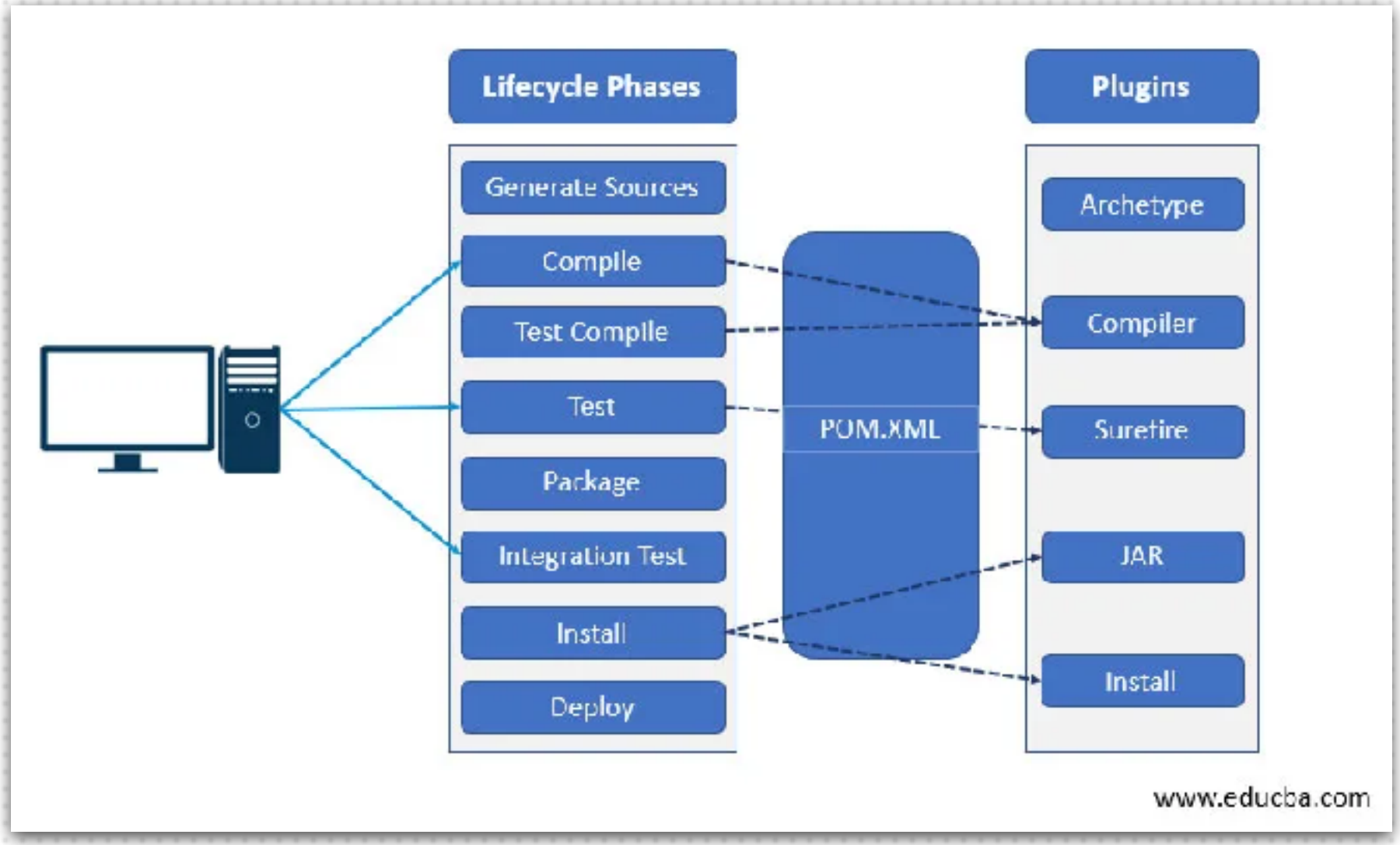
A Maven File and Directory Structure

```
<project xmlns="http://maven.apache.org/POM/4.0.0" x
     mlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
     http://maven.apache.org/xsd/maven-4.0.0.xsd">

     <modelVersion>4.0.0</modelVersion>
     <groupId>org.rebecalang</groupId>
     <artifactId>rmc</artifactId>
     <version>2.8.4</version>
     <dependencies>
          <dependency>
               <groupId>junit</groupId>
               <artifactId>junit</artifactId>
               <version>4.4</version>
          </dependency>
          <dependency>
               <groupId>org.antlr</groupId>
               <artifactId>antlr</artifactId>
               <version>3.4</version>
          </dependency>
          <dependency>
               <groupId>org.rebecalang</groupId>
               <artifactId>compiler</artifactId>
               <version>LATEST</version>
          </dependency>
     </dependencies>
</project>
```



Maven Life Cycle



Unit Testing

Bugs and Testing

- **Bugs** are inevitable in any complex software system.
  - Industry estimates: 10-50 bugs per 1000 lines of code.
  - A bug can be visible or can hide in your code until much later.

- **testing**: A systematic attempt to reveal errors.
  - Failed test: an error was demonstrated.
  - Passed test: no error was found (for this particular situation)

JUnit

- Junit is a unit testing framework for Java
- Allows you to write unit tests in Java using a simple interface
- Automated testing enables running and rerunning tests very easily and quickly

```
import org.junit.*;
import static org.junit.Assert.*;

public class TestSuiteName {
    ...
    @Test
    public void testCaseName() {
        ...
    }
}
```

Asserting the Results

```
import org.junit.*;
import static org.junit.Assert.*;

public class TestArrayIntList {
    @Test
    public void testAddGet1() {
        ArrayIntList list = ArrayIntList();
        list.add(42);
        list.add(-3);
        list.add(15);
        assertEquals(42, list.get(0));
        assertEquals(-3, list.get(1));
        assertEquals(15, list.get(2));
    }
    @Test
    public void testIsEmpty() {
        ArrayIntList list = new ArrayIntList();
        assertTrue(list.isEmpty());
        list.add(123);
        assertFalse(list.isEmpty());
    }
}
```

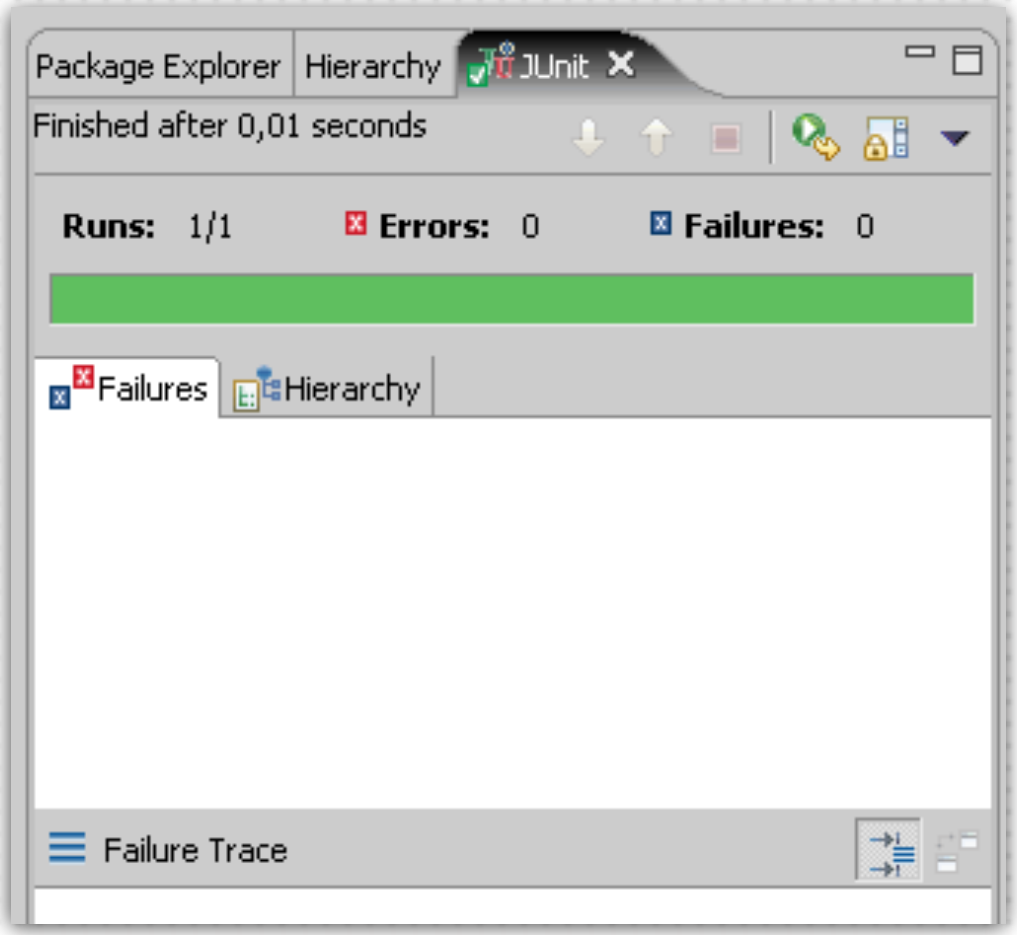
Asserting the Results

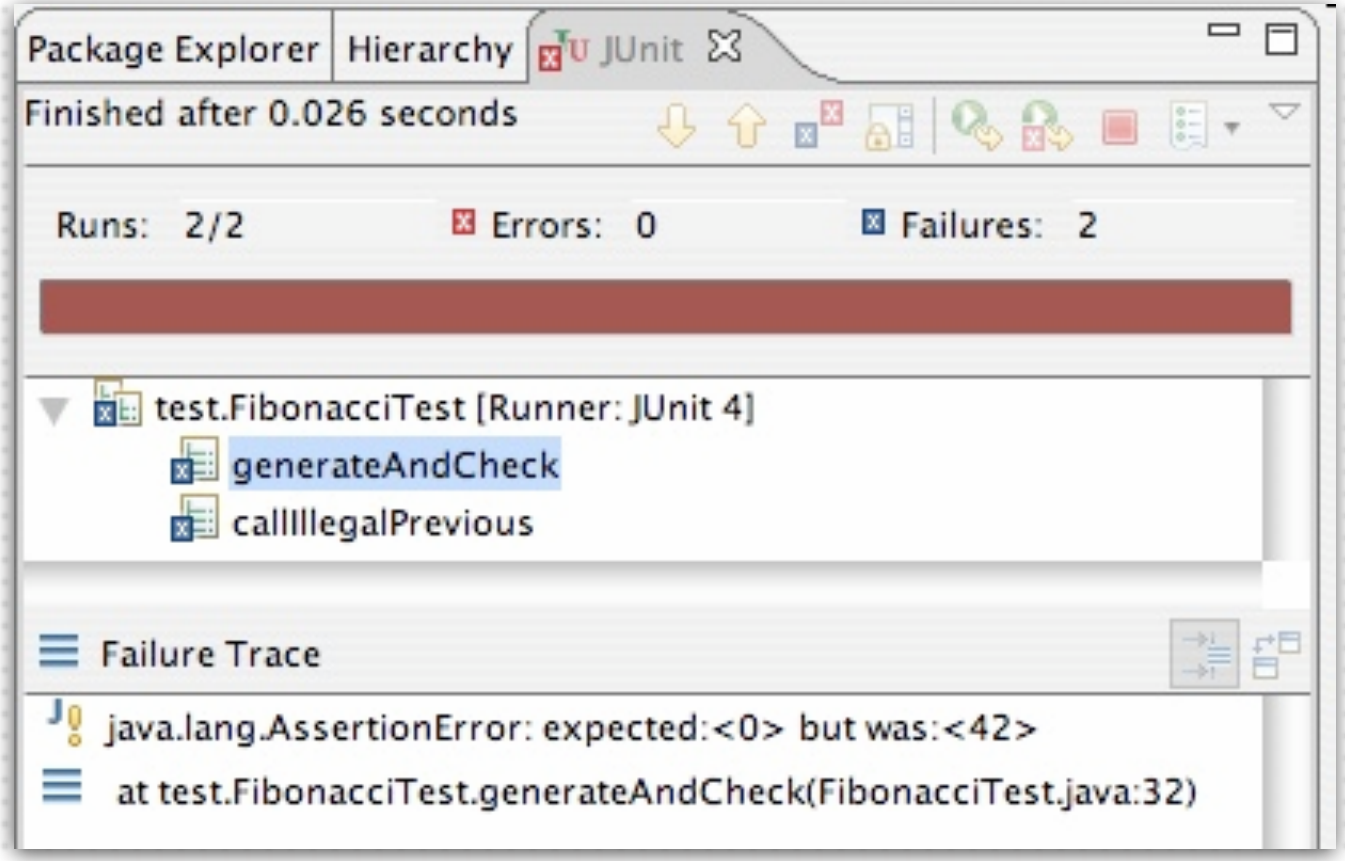
```
import org.junit.*;
import static org.junit.Assert.*;

public class TestArrayIntList {
    @Test
    public void testAdd
        ArrayIntList li
        list.add(42);
        list.add(-3);
        list.add(15);
        assertEquals(42
        assertEquals(-3
        assertEquals(15
    }
    @Test
    public void testIsE
        ArrayIntList li
        assertTrue(list
        list.add(123);
        assertFalse(list.isEmpty());
    }
}
```

`assertTrue(`**test**`)`	fails if the boolean test is `false`
`assertFalse(`**test**`)`	fails if the boolean test is `true`
`assertEquals(`**expected**`, `**actual**`)`	fails if the values are not equal
`assertSame(`**expected**`, `**actual**`)`	fails if the values are not the same (by ==)
`assertNotSame(`**expected**`, `**actual**`)`	fails if the values *are* the same (by ==)
`assertNull(`**value**`)`	fails if the given value is *not* `null`
`assertNotNull(`**value**`)`	fails if the given value is `null`
`fail()`	causes current test to immediately fail

Integrated with IDEs





Complicate Operations

- In some cases the correct result is throwing an exception

```
import org.junit.*;
import static org.junit.Assert.*;

public class TestArrayIntList {
    @Test
    public void testAddGet1() {
        ...
    }
    @Test
    void assertThrowsException() {
        String str = null;
        assertThrows(IllegalArgumentException.class, () -> {
            Integer.valueOf(str);
        });
    }
}
```

Setup and Teardown

- Perform some configurations before/after running a set of test cases
- Perform some configurations before/after each test run

```
class StandardTests {

    @BeforeAll
    static void initAll() { ... }

    @BeforeEach
    void init() { ... }

    @Test
    void SimpleTest() { ... }

    @AfterEach
    void tearDown() { ... }

    @AfterAll
    static void tearDownAll() { ... }

}
```

Version Controlling (Git)

We need Git?


BECAUSE OF ROSS' GREAT ACHIEVEMENTS, I HAVE DECIDED TO NOMINATE HIM TEAM LEADER.
I'M YOUR TEAM LEADER NOW, YOU SHALL DO AS I SAY! MY FIRST COMMAND IS TO GET RID OF GIT!
IT DOESN'T MAKE ANY SENSE... HOW ARE WE GOING TO TRACK OUR CODE?
WE DON'T NEED THE OVER-COMPLICATION OF GIT. WE CAN MANAGE OUR 2 MILLION LINES OF CODE WITH A SHARED FOLDER ON A SERVER.
#101 - "WHO NEEDS GIT?" - BY SALVATORE IOVENE, JAN. 26TH 2009
HTTP://WWW.GEEKHEROCOMIC.COM/

What is Version Controlling

- We have many files!
- Many versions of our program which is developed during the time
- Keeping track of changes in the code (wrong changes)
- Sharing the code
- What about conflicts?

Team Work!

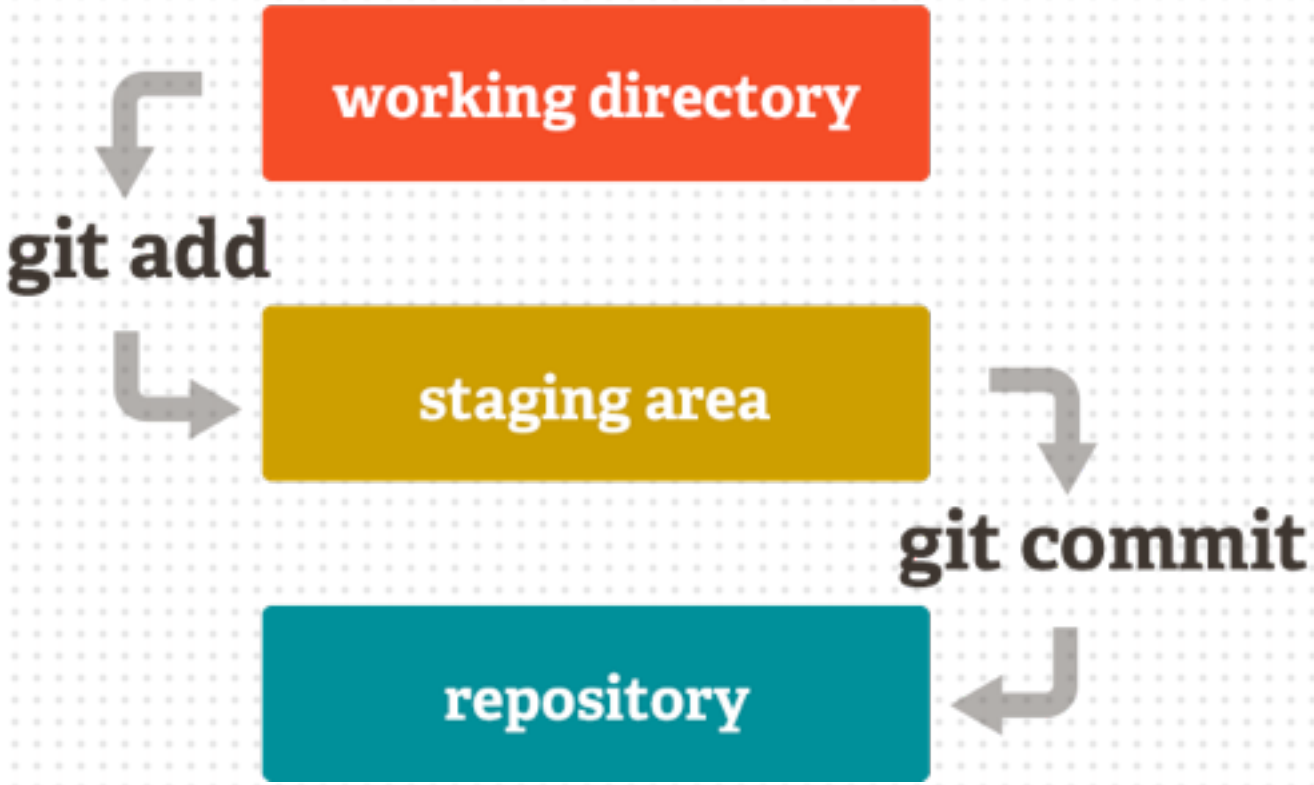
- Working in a team is not easy!
  - Code without interference
  - Go back to a previous version!
  - Integrate code of multiple developer's easily
  - Know who did what and when
- Keep your code secure



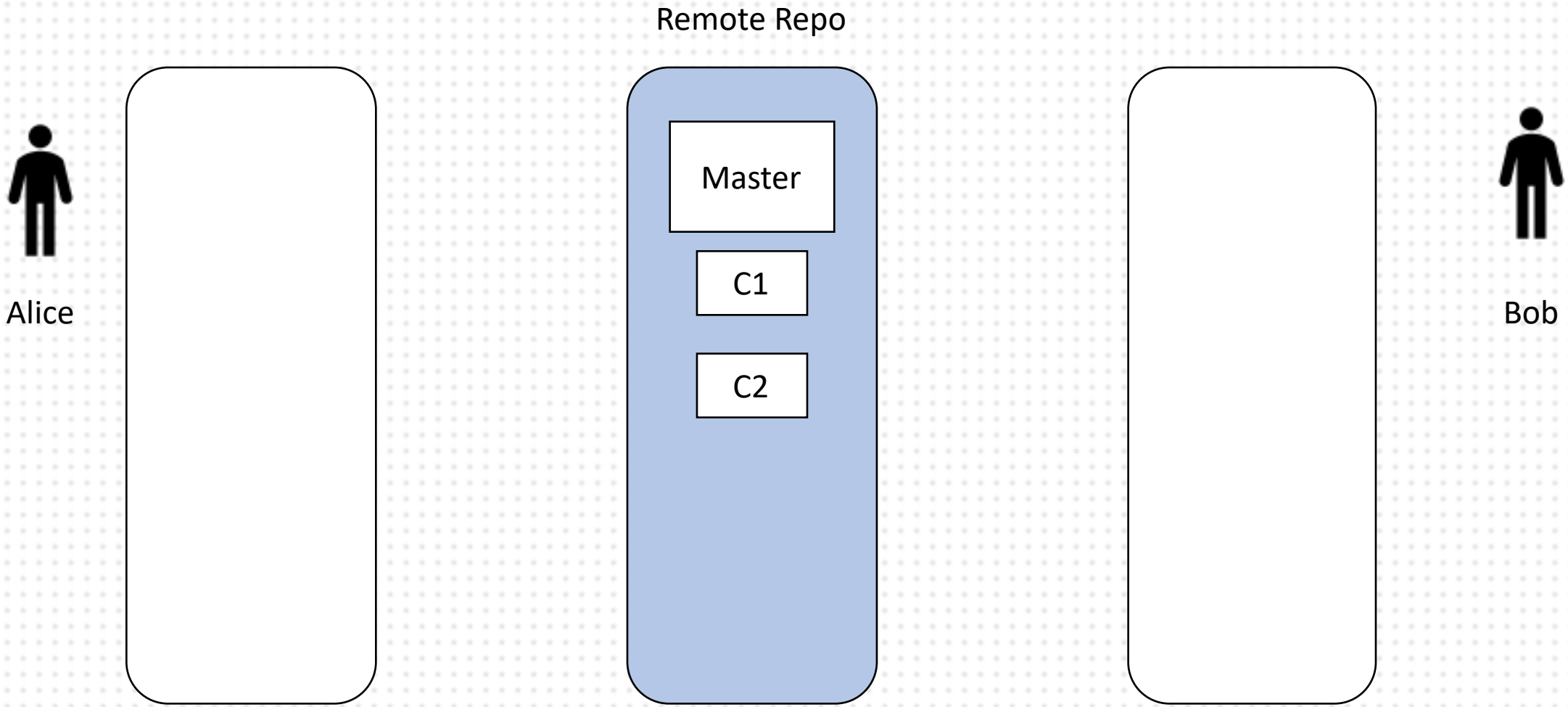
GIT

- A distributed version control system
- Command-Line Tool (accessible with Terminal on the Mac or Git Bash on Windows)
- Git is the most popular version control system in the industry, by far
- A proper and detailed understanding of Git will allow you to make a transition to any other distributed VCS easily
- Most popular VCS are similar to Git

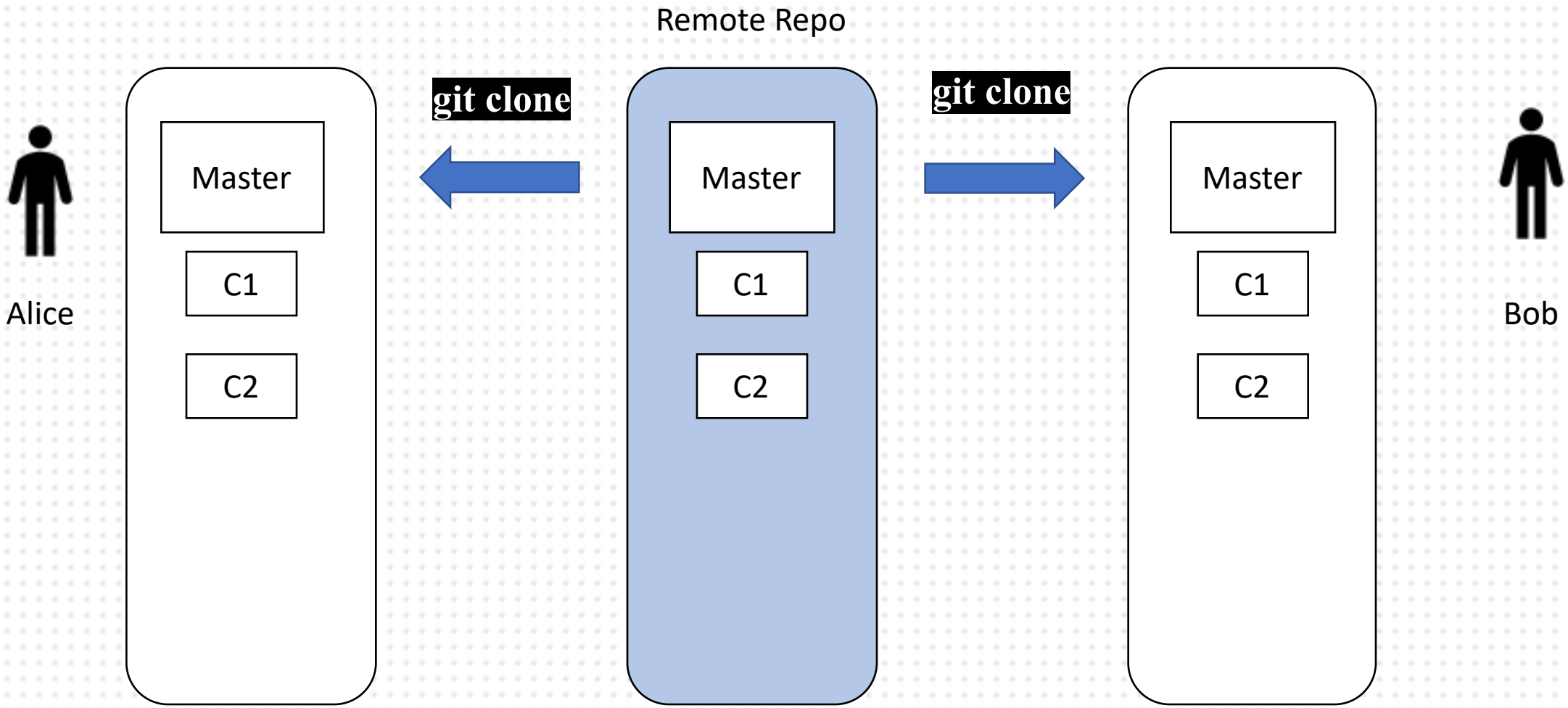
Basic Working Model of Git



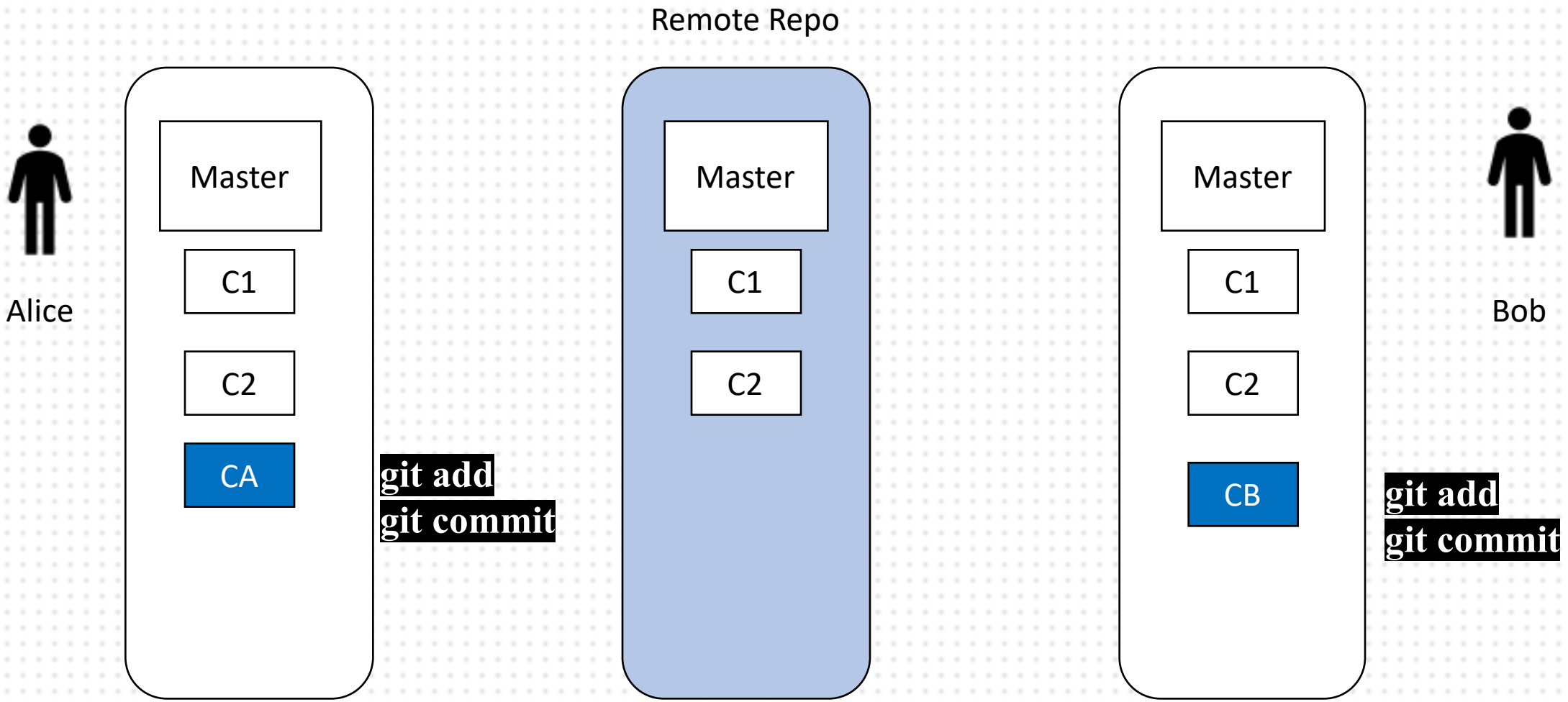
Collaboration with Git



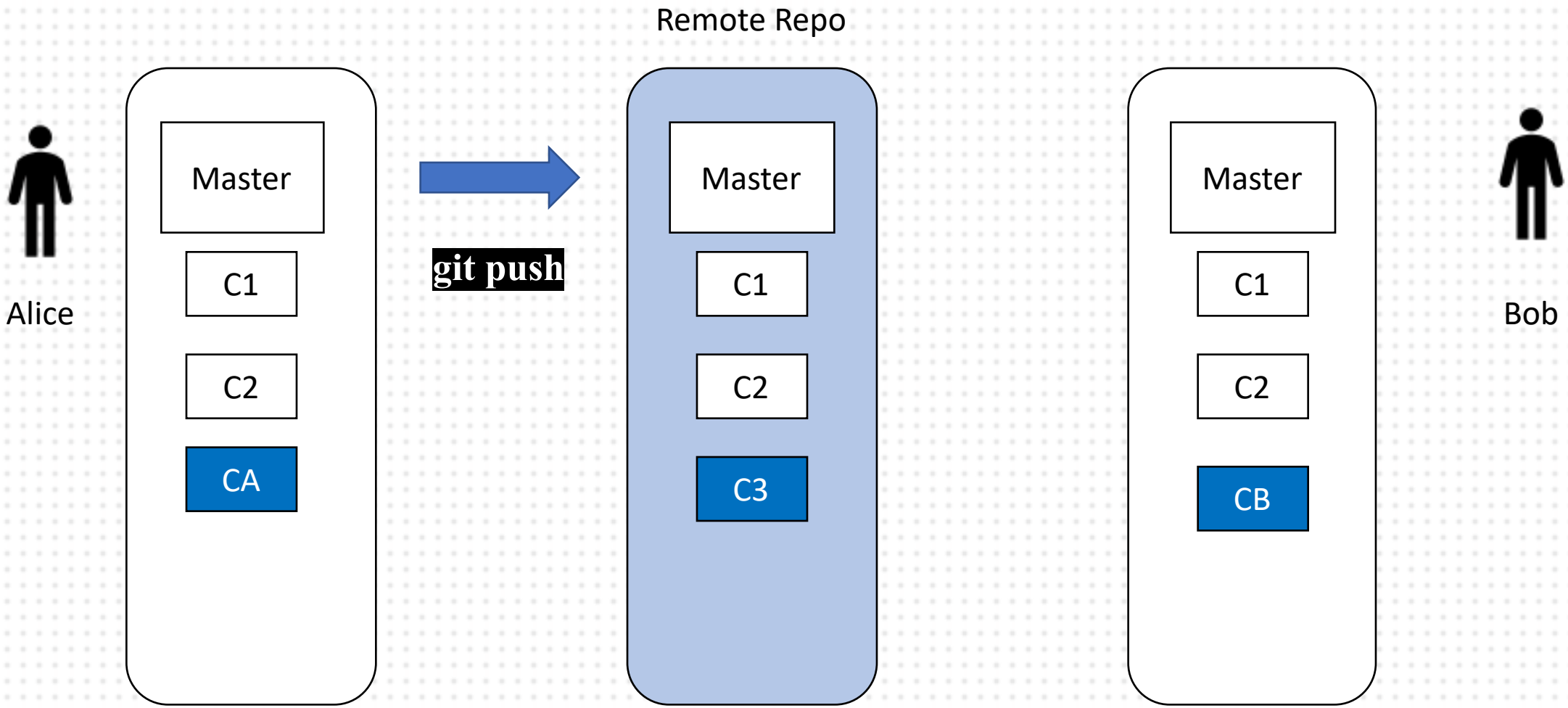
Collaboration with Git



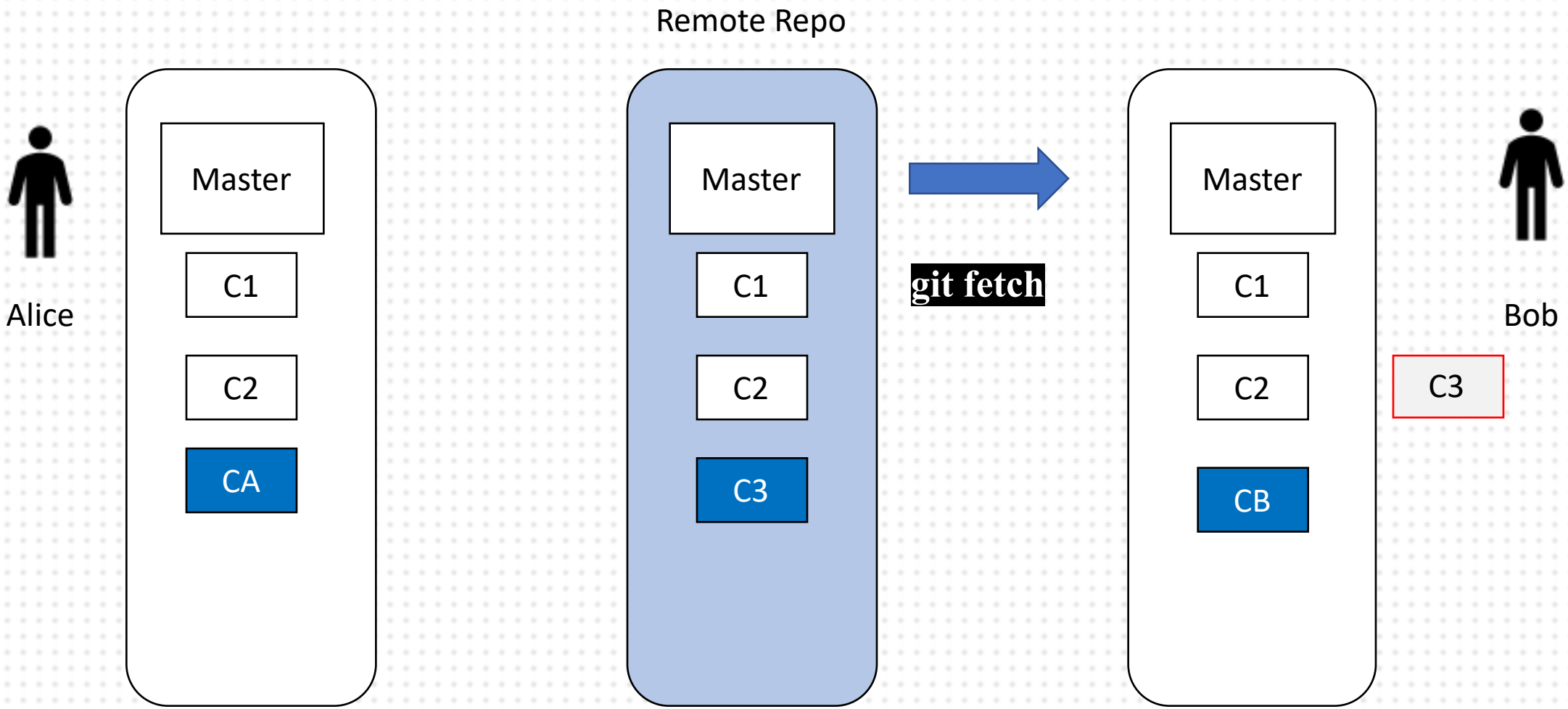
Collaboration with Git



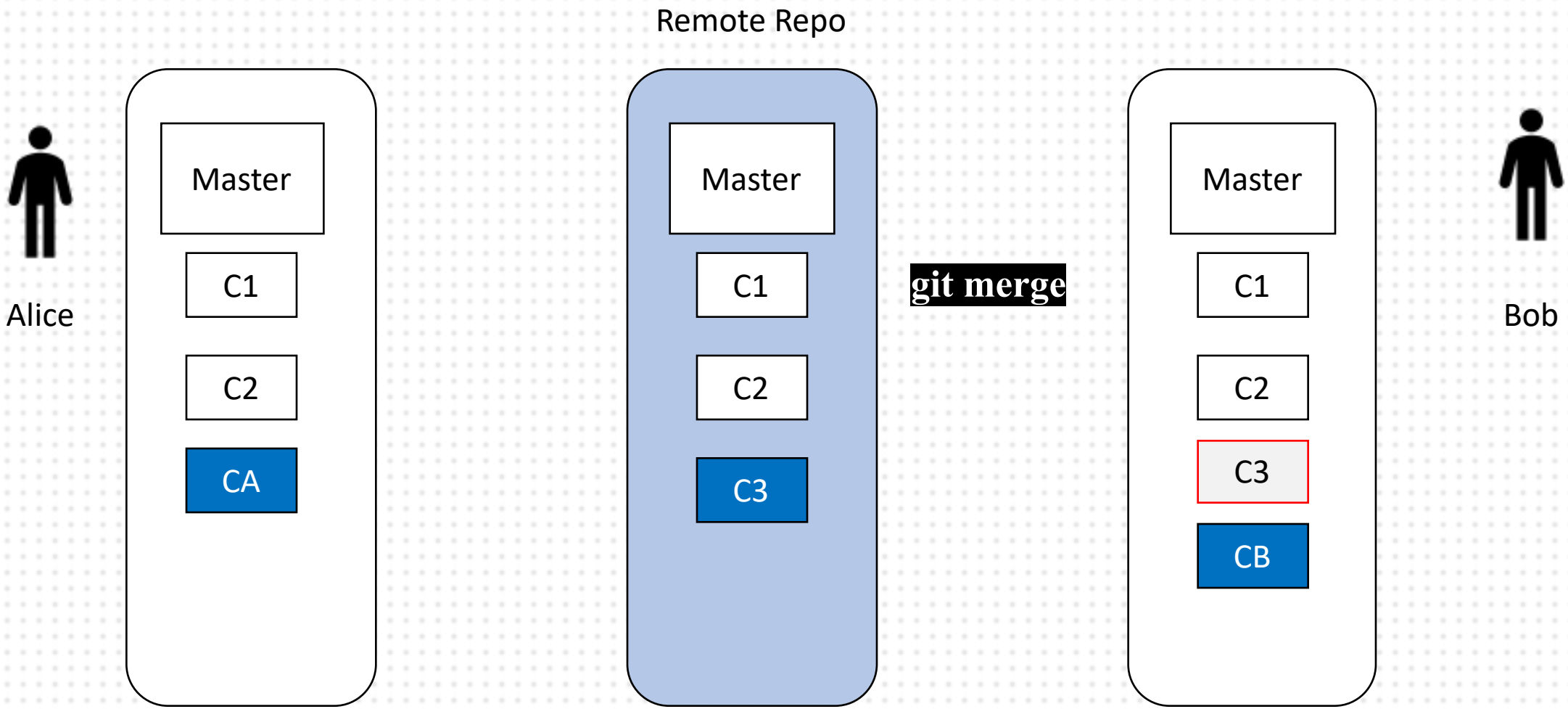
Collaboration with Git



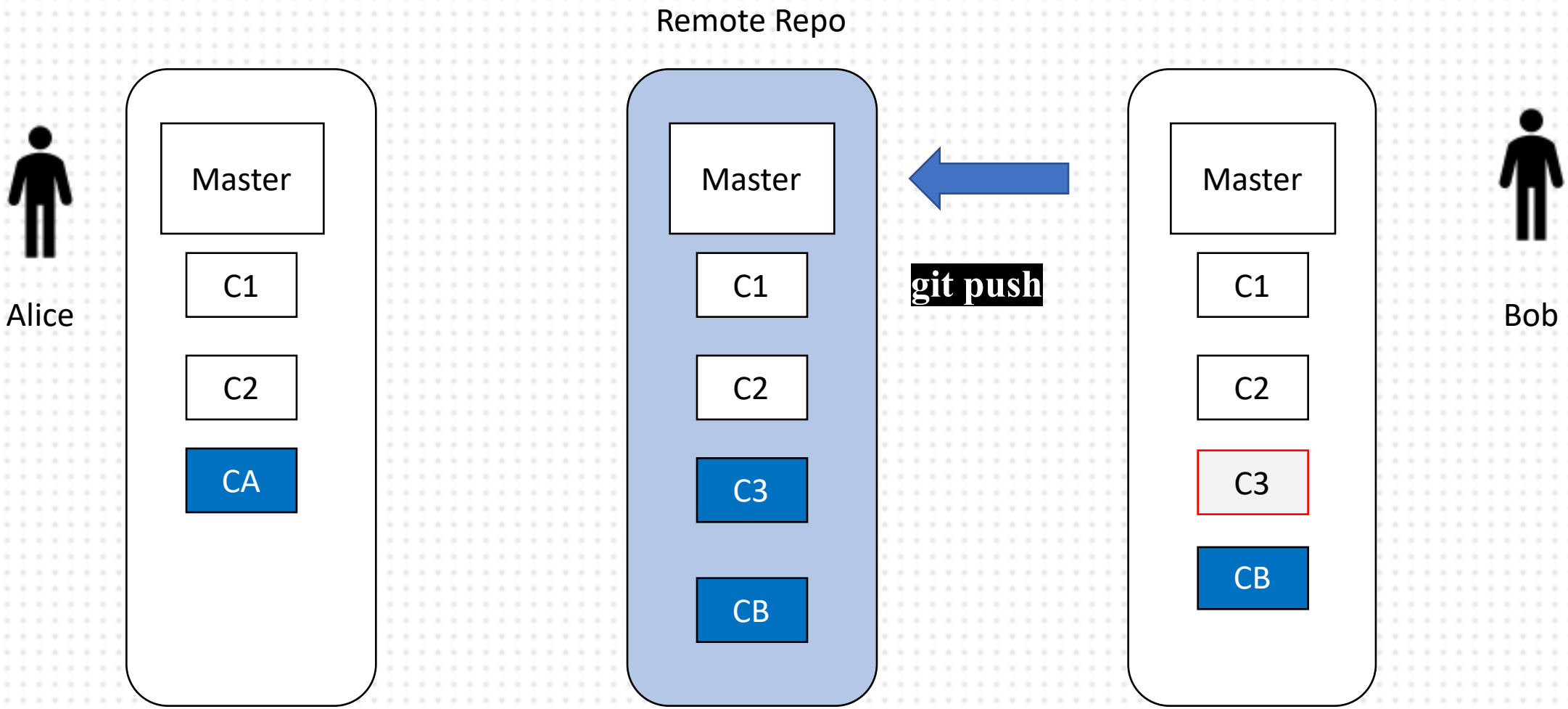
Collaboration with Git



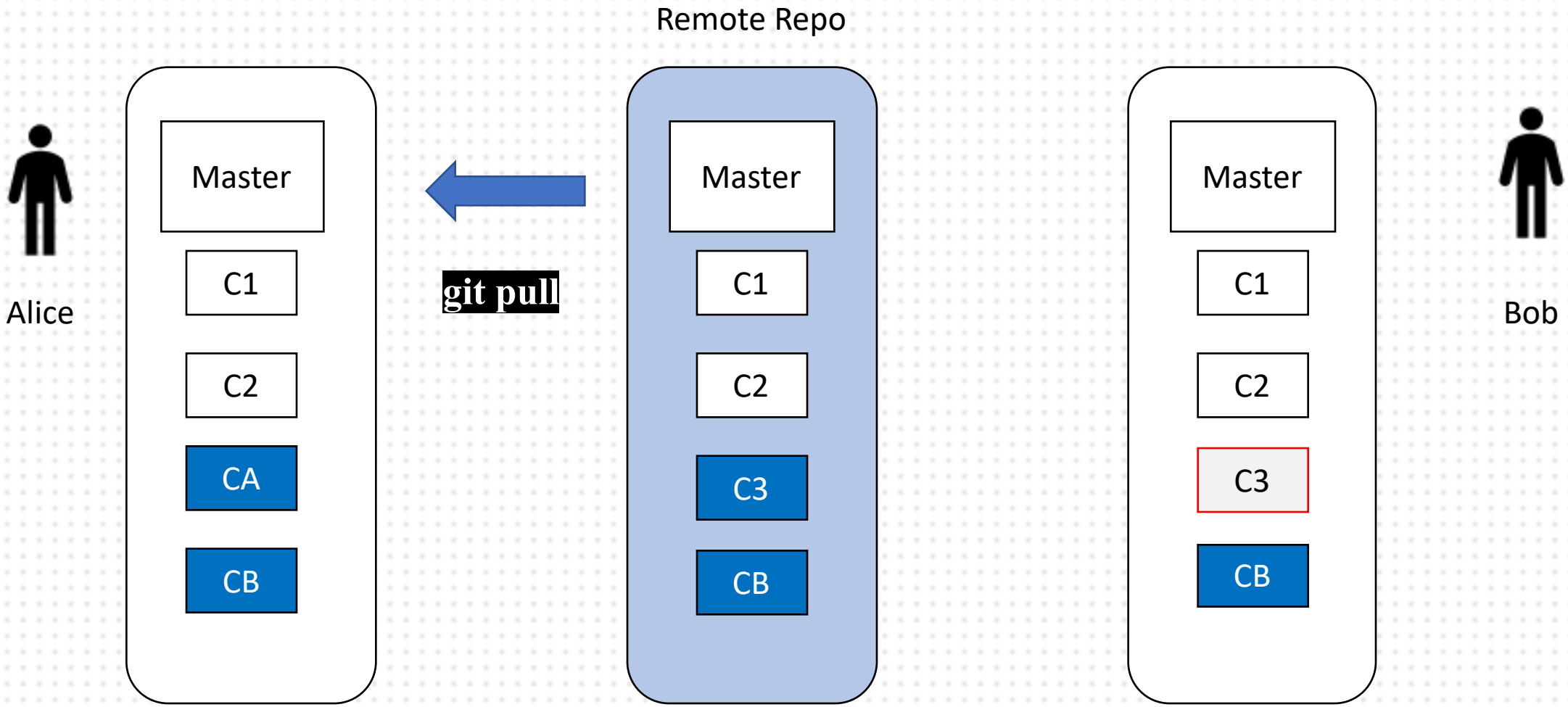
Collaboration with Git



Collaboration with Git



Collaboration with Git



Branching Strategies

